

Training High-Level Schedulers with Execution-Feedback Reinforcement Learning for Long-Horizon GUI Automation

Zehao Deng^{1,2*}, Tianjie Ju², Zheng Wu², Zhuosheng Zhang^{2†}, Gongshen Liu²

¹Soochow University

²Shanghai Jiao Tong University

2327406010@stu.suda.edu.cn, {jometeorie,wzh815918208,zhangzs,lgshen}@sjtu.edu.cn

Abstract

The rapid development of large vision-language model (VLM) has greatly promoted the research of GUI agent. However, GUI agents still face significant challenges in handling long-horizon tasks. First, single-agent models struggle to balance high-level capabilities and low-level execution capability, facing prevalent issues of responsibility coupling and capability conflicts. Second, agents lack awareness of the task state, leading to progress loss in long-horizon tasks. To address these challenges, we propose a staged execution-feedback reinforcement learning algorithm. Unlike training a unified policy model, we focus on training high-level scheduling models. Specifically, we propose and train two agents: a Coordinator, responsible for the strategic planning and task decomposition; and a State Tracker, responsible for context compression and information management to maintain the task's state and coherence. Based on this, we built the Coordinator-Executor-State Tracker (CES) multi-agent framework, which can be integrated with any low-level Executor model, assisting the Executor in solving long-horizon tasks through task scheduling and state management. Experiments on long-horizon task benchmarks demonstrate that CES significantly enhances the system's planning and state management capabilities. Furthermore, analysis confirms that our trained high-level scheduling module is a generalizable, plug-and-play module that significantly enhances the long-horizon capabilities of various Executors. Code is available at <https://github.com/hehehahi4/CES>.

1. Introduction

Graphical User Interface (GUI) agents play a crucial role in automating complex tasks [1, 4, 14, 25, 28, 31, 37, 40]. Traditional training paradigm for GUI agents primarily relies

*Work done during Zehao Deng's visit at Shanghai Jiao Tong University. † Corresponding author.

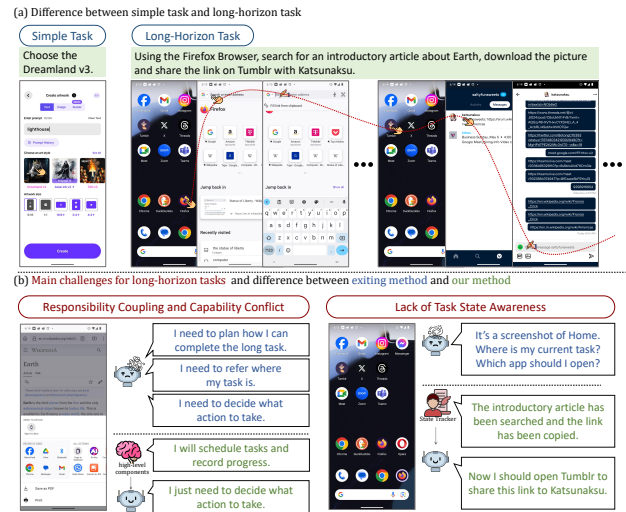


Figure 1. (a) Difference between simple tasks and long-horizon tasks. A simple task only involves a single action on one screen driven by atomic instruction, whereas a long-horizon task requires a complex trajectory driven by ambiguous, high-level user instruction. (b) Comparison of how existing methods and our method address long-horizon challenges. Left (Responsibility Coupling and Capability Conflict): A single agent is overloaded by coupling high-level capability and low-level execution. Our method resolves this by decoupling these roles into high-level and low-level components. Right (Lack of Task State Awareness): A single agent loses context on ambiguous screens like the Home screen. Our State Tracker provides high-semantic memory, enabling correct, context-aware decisions.

on Supervised Fine-Tuning (SFT), where the model learns the mapping from environmental states to actions through imitation learning [24, 35, 48]. However, SFT heavily depends on large-scale, costly, and meticulously annotated high-quality trajectory data, and often exhibits poor generalization capability in unseen environments [12, 21, 23, 33].

Recent studies [16, 23, 34, 47] have largely focused

on using rule-based Reinforcement Learning (RL) to train agents. They utilize verifiable reward functions, supplanting the need for expensive manual annotations and human feedback. RL has been demonstrated as an efficient alternative to SFT in GUI tasks, as it requires only a relatively small number of high-quality training data to achieve significant performance gains [23]. While existing methods have achieved commendable results in simple tasks [18, 21, 23], they still face two fundamental problems when confronted with long-horizon tasks, illustrated in Figure 1:

(i) **Responsibility Coupling and Capability Conflict in Single-Agent Architectures.** Current mainstream end-to-end model attempts to couple heterogeneous capabilities, such as long-term task planning, multi-step reasoning, GUI element grounding, and precise action execution, within a unified policy network. This design in optimization presents fundamental difficulties: a model with finite parameters struggles to simultaneously master high-level abilities, such as decomposing complex instructions, tracking task progress, alongside low-level abilities like grounding and execution. As task complexity increases, this coupling may lead to a catastrophic collapse of the model’s diverse capabilities.

(ii) **Lack of Task State Awareness.** In long-horizon tasks, an accurate awareness of the current progress is crucial for making correct decisions. Most methods rely on historical action sequences (e.g., `Click(x, y)`), but these low-level actions provide almost no task state information or semantic context. Therefore, the agent has to primarily rely on the visual information from screenshots to infer its progress. However, screenshots are an insufficient and unreliable representation of task progress. This limitation makes it difficult for the agent to determine its current position in a long task, leading to errors and progress loss.

To address the challenges above, we propose a staged execution-feedback RL algorithm, where the insight is to resolve responsibility coupling and conflicting optimization objectives. To support this algorithmic paradigm, we built a Coordinator-Executor-State Tracker (CES) framework, which structurally decouples the complex automation process into three specialized agents. Unlike training a unified and overloaded policy model, our algorithm is designed to optimize specific high-level scheduling models. Specifically, we treat the low-level Executor as a frozen, plug-and-play component to provide verifiable feedback, and focus our training exclusively on the Coordinator and State Tracker. Through our staged optimization strategy, the Coordinator is trained to handle strategic planning and task decomposition, effectively decoupling high-level reasoning from low-level execution; meanwhile, the State Tracker is trained to act as dynamic memory, directly solving the lack of task state awareness by maintaining a high-semantic state summary in natural language.

In summary, our main contributions are as follows:

- We build CES multi-agent framework, featuring general-purpose, plug-and-play high-level components (Coordinator and State Tracker) that can integrate with various Executors and enhance their abilities.
- We introduce a State Tracker, whose core task is dynamic context compression and state summarization, effectively resolving the state unawareness problem and maintaining the agent’s logical coherence in long-horizon tasks.
- We propose a staged execution-feedback RL strategy. The core of this algorithm is to decouple high-level capabilities from low-level execution: it freezes a pre-trained Executor and uses the reward signals it generates to exclusively train the high-level Coordinator and State Tracker.
- Extensive experiments demonstrate that our method significantly enhances the long-horizon scheduling and state management capabilities of various Executor models and surpasses existing baselines.

2. Related Work

2.1. GUI Agent

Traditional mobile intelligent assistants primarily relied on rule-based or intent-driven API calls or structured text representations [6, 13, 44]. However, these methods are often platform-specific [32], difficult to generalize [23]. With the development of VLMs, research has shifted towards pure-vision settings [13, 40, 42]. In this pure-vision paradigm, the agent receives only screenshots and task descriptions as input, generating coordinate-based actions directly in pixel space [4, 8, 10, 44].

Early methods [4, 25, 28, 35] commonly relied on the SFT paradigm, training models via imitation learning. However, the SFT paradigm has two major limitations: dependency on large-scale annotated data and poor generalization [23, 27, 29, 33, 47].

2.2. RL-Based GUI Agent

To enable agents to learn more strategic decision-making, recent research has gradually shifted to RL, which is a key paradigm for enhancing the planning and decision-making capabilities of GUI agents in difficult task scenarios [13, 21, 27]. RL involves fine-tuning pretrained VLMs using relatively small-scale, high-quality interaction trajectories, thereby enhancing their specialized reasoning and decision-making skills without compromising their original capabilities [14, 39, 43]. Inspired by the success of DeepSeek-R1 [5], the Group Relative Policy Optimization (GRPO) [26] algorithm has been widely adopted in the GUI agent domain. GUI-R1 [23] is a representative model that applies R1-style reinforcement learning to enhance the capabilities of VLMs in high-level, real-world GUI tasks.

However, these RL-based single-agent methods [18, 21–

23, 27, 29, 30], while improving the decision-making algorithm, fail to address a fundamental problem: the capability overload of a single policy network. We argue that whether using SFT or RL, coupling heterogeneous abilities like high-level strategic planning with low-level visual perception and precise execution into the same model inevitably leads to optimization conflicts and role confusion. Therefore, we innovate through architecture, thoroughly decoupling different responsibilities to enhance all facets of the agent’s capabilities via specialized division of labor and collaboration.

2.3. Multi-agent Framework

Multi-agent systems represent a key trend in the development of LLM-driven agents [45], overcoming the inherent limitations of single agents in handling complex, dynamic, and long-horizon tasks through specialized division of labor and intelligent collaboration [15, 38]. For example, Mobile-Agent-v3 [43] coordinates four agents to achieve robust, adaptive task automation with knowledge evolution capabilities. MobiAgent [45] adopts a Planner-Decider-Grounder multi-agent architecture, combined with an acceleration framework to improve execution accuracy.

However, existing works [9, 17, 43] often use general VLMs, employing prompt engineering to make models play different roles. This collaboration lacks deep optimization for each role, making it difficult to achieve optimal efficiency and specialized capability. Furthermore, existing research often overlooks the importance of task state management and information compression in long-horizon tasks, merely using historical actions [20, 27] or passively storing large amounts of redundant information [7, 11]. Our CES framework, by introducing a dedicated State Tracker and an execution-feedback RL strategy, provides specialized and efficient optimization paths for each multi-agent role.

3. Preliminary Experiment

In the Introduction, we posited a key hypothesis: GUI agents fail in long-horizon tasks due to a lack of task state awareness, arguing that screenshots are an insufficient and unreliable representation of task progress.

To empirically validate this specific hypothesis, we designed a temporal reasoning experiment. This experiment is built upon a key logic: if a screenshot were a sufficient representation of state, an agent should be able to reliably determine its relative position in a task’s timeline. Our experiment directly tests this ability by tasking three powerful GUI agents to determine the correct temporal order of two screenshots sampled from the same task trajectory.

Results in Figure 2, reveal a clear dichotomy. The model’s accuracy is high for adjacent steps, but drops dramatically as the step interval between the screenshots increases. This failure occurs for two primary reasons: First,

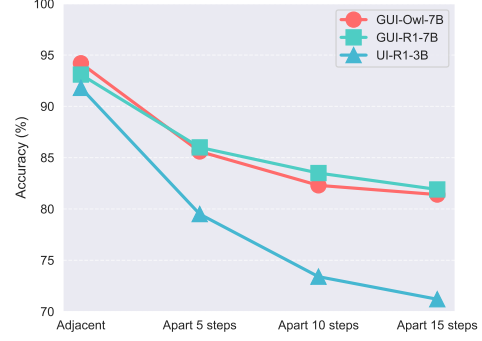


Figure 2. Temporal Judgement Accuracy. While accuracy is high for adjacent steps, it drops dramatically as the step interval increases. This result empirically demonstrates that screenshots fail to represent task state sufficiently, and we need a mechanism to record progress for long-horizon tasks.

a long-horizon task often involves repeating screenshots, such as the Home screen or going back to an interface, leading to confusion about its true progress. Second, tasks frequently navigate into Out-of-Distribution (OOD) interfaces [36], such as deep, specialized menus that the model has not encountered during training. When faced with an unfamiliar OOD screen, the model has no prior knowledge or context to determine its state in the overall task.

This failure highlights the critical need for a mechanism to explicitly maintain high-level semantic state. To address both this critical state awareness bottleneck and the challenge of responsibility coupling outlined in our introduction, we then introduce our CES framework and the staged execution-feedback RL algorithm in the subsequent section.

4. Methodology

4.1. Task Setup

We define the GUI task as a Markov Decision Process (MDP), characterized by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{F}, R \rangle$, where $\mathcal{S}$ is the state space containing all possible GUI environment states; $\mathcal{A}$ is the action space representing all executable atomic actions; $\mathcal{F} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the state transition function; and $R : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is the reward function. At each timestep t , the agent’s policy π generates a chain of thought th^t and an action $a^t \in \mathcal{A}$ based on the user-specified natural language instruction q , the current screen state $s^t \in \mathcal{S}$, and the historical action information H^t , that is $(th^t, a^t) \sim \pi(\cdot | q, s^t, H^t)$. Our goal is to learn an optimal policy π^* that maximizes the cumulative reward. However, directly learning an end-to-end policy π faces significant challenges. Therefore, we decompose this decision-making process into a structured multi-agent collaborative loop, transforming the original MDP problem into a series of more manageable sub-problems.

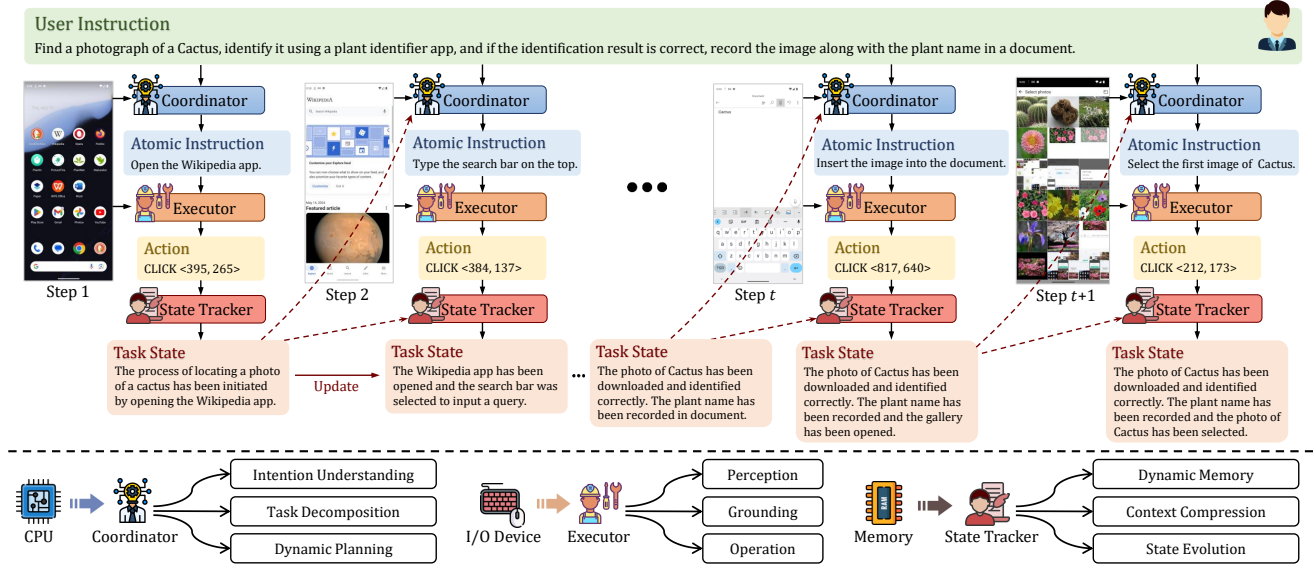


Figure 3. The CES multi-agent loop framework. CES executes complex long-horizon tasks through the collaboration of three specialized agents. The Coordinator, as the task scheduling and decision-making core, combines the user’s high-level instruction and the current task state (provided by the State Tracker) to decompose the task into a clear atomic instruction. The Executor, acting as the tool, precisely executes this atomic instruction and interacts with the GUI environment. Finally, the State Tracker, as the memory, observes the Executor’s output and updates it into a high-semantic task state summary, which is then fed back to the Coordinator for the next step of decision-making.

4.2. CES Collaborative Loop

As shown in Figure 3, our CES framework operates as a loop information flow: the Coordinator plans based on the state from the State Tracker, the Executor executes the instruction, the State Tracker updates the state based on the execution result, and the Coordinator receives the new state for the next planning step. This division of labor is inspired by a modern operating system, where the Coordinator acts as the “CPU” (planning), the Executor as the “I/O Device” (action), and the State Tracker as the “Dynamic Memory” (state management). Through this OS-inspired design, we completely separate high-level strategic planning from low-level precise execution, allowing each agent to focus on its core capabilities and jointly accomplish complex long-horizon tasks through efficient collaboration.

4.2.1. Coordinator

The Coordinator is the strategic decision-making core of the task. Its primary responsibility is to translate the user’s high-level, often ambiguous, natural language instructions into a series of clear, executable steps. To make decisions that align with the long-term strategy yet adapt to the current situation, the Coordinator fuses three types of information at each timestep t : the user’s high-level instruction q , which serves as the global macro-objective, the compressed high-semantic state summary m^{t-1} provided by the State

Tracker, and the current screenshot s^t . Formally,

$$l^t = \pi_c(q, m^{t-1}, s^t), \quad (1)$$

where π_c denotes the Coordinator agent’s policy, and l^t is the generated atomic instruction, which focuses on guiding the Executor on the specific action to perform in the current environment. Furthermore, when exceptions or errors occur, the Coordinator can reflect and re-plan the task, guiding the Executor to take corrective actions and return to the correct path.

4.2.2. Executor

The Executor is the action endpoint in the CES framework. Its role is strictly limited to translating the atomic instruction l^t from the Coordinator into a physical operation on the interface. The core advantage of this design is the complete separation of cognitive load: the Executor does not need to understand the user’s long-term intent or maintain complex task context. Its sole task is to find the target element described by l^t on the current screen s^t and perform the corresponding action. Formally,

$$u^t = (th^t, a^t) = \pi_e(l^t, s^t), \quad (2)$$

where u^t is the Executor’s output, including the chain of thought th^t and the standardized action a^t , and π_e is the Executor’s policy.

4.2.3. State Tracker

The State Tracker is key to solving the long-horizon challenges for GUI agents. Its core function is dynamic context compression and state updating. It acts as the framework’s dynamic memory unit, shifting the agent’s state understanding task from processing high-dimensional, redundant historical screenshots to a low-dimensional, high-semantic natural language space. It is a language model that does not directly perceive the GUI environment but rather infers and generates the new state summary m^t by understanding the Executor’s output u^t , combined with the user intent q and the previous task state m^{t-1} :

$$m^t = \pi_s(q, m^{t-1}, u^t), \quad (3)$$

where π_s is the State Tracker’s policy. This natural language-based state evolution not only compresses information and filters visual noise but also provides an exceptionally clear and coherent basis for the Coordinator’s decisions, effectively solving context loss in long-horizon tasks.

4.3. Staged Execution-Feedback Reinforcement Learning

We post-train the Coordinator on vision-language model and post-train the State Tracker on language model. Notably, in our framework, the Executor is designed as a frozen, swappable component. We do not train it; instead, we directly use a fixed, powerful pretrained GUI model. This design allows us to focus our research on optimizing the Coordinator’s planning and the State Tracker’s state understanding, avoiding interference from low-level execution details. This also demonstrates the generality and plug-and-play nature of the CES framework, which can be combined with any powerful execution model as its brain to enhance long-horizon task capabilities.

4.3.1. Warm-up SFT

Firstly, we perform warm-up SFT for the Coordinator and State Tracker. This aims to let each agent learn its basic role, responsibilities, and strict output format. We use trajectories from existing datasets and, through automated scripts, we construct high-quality (input, output) data pairs for the preliminary fine-tuning of both agents.

4.3.2. Rule-based RL

GRPO. To further enhance the model’s generalization, we use RL to optimize the models, learning a planning policy that maximizes long-term task success. We use the GRPO algorithm [26]:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \min(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1 \pm \epsilon) \hat{A}_i) \right] - \beta D_{KL}(\pi_\theta || \pi_{ref}), \quad (4)$$

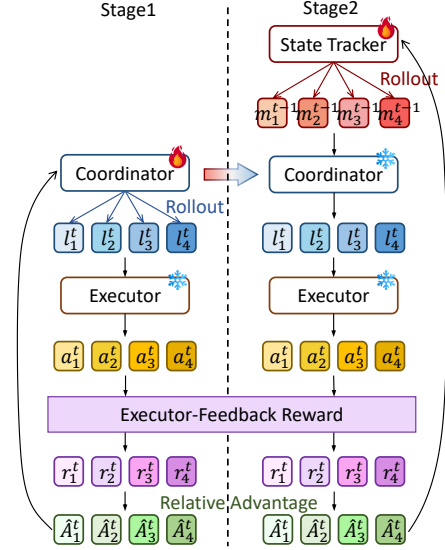


Figure 4. Our proposed staged execution-feedback RL strategy. This strategy utilizes the Execution-Feedback Reward from a fixed Executor to sequentially optimize the Coordinator (Stage 1) and State Tracker (Stage 2) in two independent training phases.

where $\rho_i(\theta) = \frac{\pi_\theta(o_i)}{\pi_{\theta_{old}}(o_i)}$ is the importance sampling ratio, π_θ and $\pi_{\theta_{old}}$ denote the current and previous policies, and o_i is the candidate output. ϵ is the clipping hyperparameter, and β controls the KL penalty against a reference policy π_{ref} .

For each input, the model generates N candidate outputs $\mathcal{O} = \{o_1, o_2, \dots, o_N\}$, and each output is scored by a rule-based reward function to get $\mathcal{R} = \{r_1, r_2, \dots, r_N\}$. The estimated relative advantage $\hat{A}_i$ is then calculated as:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_N\})}{\text{std}(\{r_1, r_2, \dots, r_N\})}. \quad (5)$$

Execution-Feedback Reward. Assigning rewards for agents responsible for abstract tasks (like planning) is difficult. Directly evaluating the quality of the Coordinator’s atomic instructions or the State Tracker’s state summaries lacks an objective standard and cannot guarantee their contribution to the final task success. To address this challenge, we do not directly evaluate the intermediate outputs. Instead, these outputs are passed through the CES loop to the Executor. The action output by the Executor is then objectively scored using a verifiable, rule-based reward function. This reward signal, originating from the final execution, is back-propagated to optimize the policies of the Coordinator and State Tracker.

Specifically, we design a rule-based reward function, termed the **Execution-Feedback Reward**:

$$R = \alpha_1 R_{format} + \alpha_2 R_{executor}, \quad (6)$$

Table 1. Long-horizon task performance comparison on three benchmarks. **Bold** indicates the best, underline indicates the second best. * indicates that the data of the benchmark is from GUI-R1’s original paper.

Model	Method	AITZ			AMEX			GUI-Odyssey		
		Type	GR	SR	Type	GR	SR	Type	GR	SR
GPT-4o	Zero Shot	—	—	—	—	—	—	37.50	14.17	5.36*
Qwen2.5-VL-7B	Zero Shot	34.23	55.27	18.11	59.52	48.24	35.10	55.60	37.78	34.37*
OS-Atlas-7B	SFT	38.52	44.14	25.97	55.11	40.30	33.89	60.42	39.74	26.96*
UI-R1-3B	RL	41.63	49.27	24.55	60.23	41.78	35.81	52.16	34.46	32.49*
GUI-Owl-7B	RL	53.86	52.08	32.70	61.56	48.38	40.48	60.60	45.96	35.82
SWIRL	Multi-Agent	—	—	—	—	—	—	<u>74.87</u>	66.39	<u>51.65</u>
GUI-R1-7B	RL	52.73	54.92	30.59	67.26	57.12	43.69	65.49	43.64	38.79*
+ GPT-5	Multi-Agent	<u>62.50</u>	<u>59.10</u>	40.55	<u>72.80</u>	52.15	35.80	70.37	49.92	42.47
+ CES (Ours)	Multi-Agent	64.44	64.58	43.05	77.57	61.64	48.48	79.24	<u>63.82</u>	53.69

Table 2. Main results evaluating CES framework’s Effectiveness and Generalization. **Baseline**: The model is used directly. **CES-P**: The same base model acts as all three roles within the CES framework via Prompting. **CES**: Our full framework, using our specialized Coordinator and State Tracker, with the base model serving as the Executor. **Bold** indicates the best.

Model	Setting	AMEX			GUI-Odyssey		
		Type	GR	SR	Type	GR	SR
UI-R1-3B	Baseline	60.23	41.78	35.81	52.16	34.46	32.49
	CES-P	42.52 (-17.71)	52.15 (+10.37)	29.12 (-6.69)	25.81 (-26.35)	53.05 (+18.59)	14.44 (-18.05)
	CES	70.39 (+10.16)	66.28 (+24.50)	43.38 (+7.57)	66.37 (+14.21)	64.29 (+29.83)	38.04 (+5.55)
GUI-Owl-7B	Baseline	61.56	48.38	40.48	60.60	45.96	35.82
	CES-P	69.18 (+7.62)	53.05 (+4.67)	44.91 (+4.43)	66.03 (+5.43)	52.15 (+6.19)	37.53 (+1.71)
	CES	75.72 (+14.16)	61.19 (+12.81)	47.24 (+6.76)	74.87 (+14.27)	61.39 (+15.43)	46.65 (+10.83)
GUI-Owl-32B	Baseline	69.13	53.24	43.16	67.15	47.33	39.60
	CES-P	81.02 (+11.89)	63.80 (+10.56)	56.34 (+13.18)	76.90 (+9.75)	65.10 (+17.77)	53.88 (+14.28)
	CES	78.55 (+9.42)	63.11 (+9.87)	52.05 (+8.89)	79.58 (+12.43)	65.42 (+18.09)	56.75 (+17.15)

where R_{format} is the format reward, rewarding the model for outputting in the $\langle think \rangle$ and $\langle answer \rangle$ tags, $R_{executor}$ is the executor reward, and α_1, α_2 are coefficients. By passing the Coordinator’s output and the current screenshot s^t to the executor, we derive the executor reward from its action:

$$R_{executor} = \gamma_1 R_{type} + \gamma_2 R_{param}, \quad (7)$$

where γ_1, γ_2 are coefficients, R_{type} rewards the correct action type, and R_{param} rewards the correct action parameters. More details can be found in Appendix A.

Staged Optimization. Based on the GRPO algorithm and the Execution-Feedback Reward, we optimize the Coordinator and State Tracker in two distinct stages, as shown in Figure 4. This strategy ensures that high-level planning and state tracking are always optimized towards goals that are verifiably effective and easily understood by the Executor.

Stage 1: Optimizing the Coordinator’s Planning Capability. In this stage, we use a frozen Executor model and

calculate the reward based on its execution results to update the Coordinator’s policy network. Since we do not yet have a trained State Tracker, the required task state m^{t-1} is sourced directly from the ground-truth annotated states in our preprocessed dataset. This ensures the Coordinator focuses on learning the mapping from a “perfect” state to the optimal atomic instruction.

Stage 2: Optimizing the State Tracker’s State Evolution Capability. After the Coordinator is trained, we freeze its parameters. In this stage, the State Tracker’s capability is the focus of optimization. The task state m^{t-1} it generates is passed through the fixed Coordinator and Executor, influencing the entire decision chain. The final Execution-Feedback Reward from the Executor is used exclusively to optimize the State Tracker, teaching it to generate the most valuable state information that helps the Coordinator make optimal decisions. In this way, the State Tracker is explicitly trained to produce state summaries that are maximally useful to the fixed Coordinator’s policy, effectively learning to generate what the Coordinator understands best.

5. Experiments

5.1. Settings

Training Details. We choose Qwen2.5-VL-7B [2] as the base model for the Coordinator and Qwen3-4B [41] for the State Tracker. For the warm-up SFT, we use the LLaMA Factory framework and trained for 1 epoch with a learning rate of $5e-5$ to prevent overfitting. For RL, we use the Verl framework to train the Coordinator for 10 epochs with a learning rate of $1e-6$ and State Tracker for 5 epochs. We use GUI-R1-7B [23] as the Executor to calculate the reward function when training. For the reward coefficients, we set α_1, α_2 to 0.1 and 0.9, respectively, and γ_1, γ_2 to 0.2, and 0.8, respectively. All experiments are conducted on $8 \times 80G$ GPUs.

Benchmarks. To thoroughly evaluate our framework’s ability to solve long-horizon tasks, we leverage three benchmarks for long-horizon, complex tasks, including AITZ [46], AMEX [3] and GUI-Odyssey [19], with an average steps of 7.5, 12.8 and 15.3 respectively. We only use the high-level instructions provided by the datasets, not the low-level ones.

Evaluation Metrics. Following previous work [20, 23, 35], we use three common metrics for GUI agent evaluation: (i) **Type**, the prediction accuracy of the action type. (ii) **GR** (Grounding), the click point prediction accuracy, where a prediction is considered correct if the point falls within the ground-truth bounding box. (iii) **SR** (Success Rate), correct only if both action type and parameters are correct. Parameter correctness includes: the point being in the bounding box, correct scroll direction, or an F1 similarity > 0.5 for input text.

5.2. Main Results

5.2.1. Long-Horizon Task Performance

As shown in Table 1, CES achieves compelling performance on complex GUI tasks by implementing atomic instruction and progress state management. On top of the GUI-R1-7B executor baseline, our method improves the Type accuracy by an average of 10.38% across all benchmarks. This significant gain strongly validates the effectiveness of our decoupled design: by having the Coordinator bear the cognitive load of high-level planning and generate clear atomic instructions, the Executor no longer needs to reason about complex task context and can focus on simple perception and localization, thereby greatly enhancing its execution accuracy and stability.

Furthermore, when we use GPT-5 via prompting as the Coordinator and State Tracker, the performance only shows a minor improvement, exemplified by an average increase

of only 4% in Type accuracy, while some metrics even degrade. This indicates that while powerful general-purpose models can offer some improvement through prompt engineering, the effect is unstable. In contrast, our CES framework, through targeted execution-feedback reinforcement learning, enables the Coordinator and State Tracker to learn planning and state understanding strategies that are much better aligned with GUI tasks, thus achieving stable and significant performance superiority.

5.2.2. Effectiveness and Generalization

To validate the efficiency and generality of our CES framework, we conducted a further experiment using three models of varying sizes. For each model, we tested three distinct configurations: (i) **Baseline**: The model is used directly. (ii) **CES-P**: The same base model acts as all three roles within the CES framework via prompting. (iii) **CES**: Our full framework, using our specialized Coordinator and State Tracker, with the base model serving as the Executor. We conducted extensive evaluations on two AMEX and GUI-Odyssey with long average task steps, with the results shown in Table 2.

From these results, we derive several key insights:

(i) **Architectural Superiority: Solving State Awareness and Cognitive Load.** A consistent finding across all model sizes is that the CES-P setup significantly outperforms the baseline. The baseline model fails in long-horizon tasks because it lacks a mechanism to manage task state, forcing it to infer progress from ambiguous screenshots. The CES framework solves this by introducing a State Tracker, which provides explicit, high-semantic memory, and a Coordinator, which decouples the cognitive load of planning from the immediate burden of execution.

(ii) **Validating the Capability Conflict Hypothesis.** While CES-P helps all models, the magnitude of the improvement reveals evidence of capability conflict while training. For the small UI-R1-3B, the CES-P setup actually resulted in a performance degradation, with a significant drop of 18.05% in SR on the GUI-Odyssey. In contrast, the large GUI-Owl-32B sees a massive improvement of 13.18% on AMEX. This suggests that the 3B model, due to its limited parameter capacity, struggled to learn these distinct planning and execution capabilities. The training-time conflict was too severe for it to be overcome. Conversely, the 32B model learned planning, execution, and state understanding abilities during training, even though these abilities remained coupled. The CES-P setup reveals the latent high-level skills that the 32B model had already acquired tentatively. This validates our hypothesis that it is extremely difficult to train these conflicting capabilities within a single policy network, and this challenge is magnified in models with smaller parameter counts.

(iii) **The Efficacy of Specialized, Trained Components.** Comparing the CES-P to CES highlights the sig-

Table 3. Ablation study on components and training stages.

Model	AMEX			GUI-Odyssey		
	Type	GR	SR	Type	GR	SR
CES	77.57	61.64	48.48	79.24	63.82	53.69
w/o Coordinator	62.63	50.57	33.27	70.11	48.03	39.15
w/o State Tracker	70.62	55.70	42.08	73.34	45.10	42.52
w/o RL (SFT only)	72.37	53.47	36.54	72.18	51.33	42.89

nificant value of our training strategy. For instance, on GUI-Odyssey, our specialized training boosts the GUI-Owl-32B’s SR from 39.60% to 56.75%, and the GUI-Owl-7B’s SR from 37.53% to 46.65%. This demonstrates that our Coordinator and State Tracker via staged execution-feedback RL are highly effective and vastly superior to simply prompting a non-specialized model for these roles. Even, we can get the comparable effect as the Gui-Owl-32B model just by using 7B Coordinator and 4B State Tracker. This also proves the effectiveness of decoupling training.

(iv) **Generality of the CES Framework.** Finally, our full framework, CES, provides a substantial and consistent performance improvement over the baseline for every executor model tested. This confirms that CES is a robust and generalizable plug-and-play solution that can effectively enhance any underlying executor, dramatically improving its long-horizon task automation capabilities.

5.3. Analysis

Component and Training Ablation. We removed the Coordinator, State Tracker, and the RL stage from our framework to observe the effects. As shown in Table 3, when we remove the Coordinator and feed the user’s high-level instruction directly to the Executor, performance drops significantly, such as SR drops by 12.77% on GUI-Odyssey. This indicates that the Executor still struggles to understand high-level instructions and make current decisions, proving the necessity of decoupling planning from execution. When the State Tracker is removed, where we instead record the last four action histories as most methods and input them to the Coordinator, performance also drops significantly. This shows the critical role of information compression and state management in long-horizon tasks; without the concise, high-semantic summary from the State Tracker, the Coordinator cannot make correct plans. Finally, when we test the model using only SFT warm-up, its performance is far below the final RL-optimized model. This demonstrates that while SFT allows agents to learn basic roles and output formats, execution-feedback RL is an indispensable step to acquire a generalizable, optimal planning policy.

Failure Case Analysis. To understand the source of our CES framework’s advantages, we conducted a detailed failure case analysis, with results shown in Figure 5. Specif-

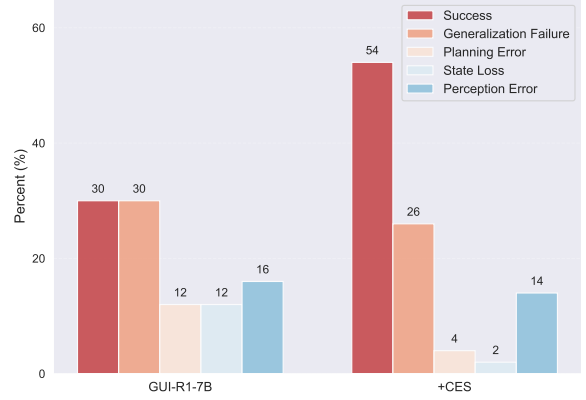


Figure 5. Failure Case Analysis. Compared to the baseline, our CES framework almost completely eliminates cognitive errors like State Loss and Planning Error.

ically, the CES framework almost completely eliminated State Loss errors, reducing the count from 14% to 2% and significantly reducing Planning Error from 12% to 4%. This result demonstrates that our framework’s performance gain stems precisely from our specially trained Coordinator and State Tracker, which successfully solve the core challenges of planning and state management. In contrast, the errors attributable to the frozen Executor, namely Perception Error and Generalization Failure, remained largely unchanged. The performance bottleneck has now effectively shifted to the inherent perceptual limitations of Executor itself. More case studies can be found in Appendix D.

6. Conclusion

In this paper, we propose a staged execution-feedback RL strategy to address the core challenges of long-horizon GUI automation. Our algorithm confronts the capability overload and lack of task progress awareness by shifting the training strategy. Instead of training a unified policy, our method thoroughly decouples high-level strategic planning from low-level precise execution. It leverages verifiable results to efficiently optimize high-level agents responsible for scheduling. Critically, our algorithm trains a dedicated State Tracker that resolves the agent’s difficulties in long-horizon tasks via dynamic context compression and high-semantic state summarization. This algorithmic approach is instantiated in our CES framework. Extensive experiments on challenging long-horizon GUI benchmarks demonstrate that our trained high-level modules are a generalizable, plug-and-play solution that significantly enhances the long-horizon planning and state management capabilities of various Executor models. In the future, synergetic evolution and joint training of multi-agent system for GUI tasks may be a promising direction.

Acknowledgment

This work was supported by the National Key R&D Program of China (No.2024YFC3306500), National Natural Science Foundation of China (62406188), and Shanghai Municipal Science and Technology Major Project (2025SHZDZX025G08).

References

- [1] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents, 2025. 1
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhao-hai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL Technical Report, 2025. 7
- [3] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Shuai Ren, and Hongsheng Li. AMEX: Android Multi-annotation Expo Dataset for Mobile GUI Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2138–2156, 2025. 7
- [4] Pengzhou Cheng, Zheng Wu, Zongru Wu, Aston Zhang, Zhuosheng Zhang, and Gongshen Liu. OS-Kairos: Adaptive Interaction for MLLM-Powered GUI Agents, 2025. 1, 2
- [5] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qishi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wan-jia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025. 2
- [6] Jingru Fan, Yufan Dang, Jingyao Wu, Huatao Li, Runde Yang, Xiyuan Yang, Yuheng Wang, Zhong Zhang, Yaxi Lu, Yankai Lin, et al. Appcopilot: Toward general, accurate, long-horizon, and efficient mobile agent. *arXiv preprint arXiv:2509.02444*, 2025. 2
- [7] Xinzge Gao, Chuanrui Hu, Bin Chen, and Teng Li. Chain-of-Memory: Enhancing GUI Agents for Cross-Application Navigation, 2025. 3
- [8] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, Yue Wen, Jingya Dou, Fei Tang, Jinzhen Lin, Yulin Liu, Zhenlin Guo, Yichen Gong, Heng Jia, Changlong Gao, Yuan Guo, Yong Deng, Zhenyu Guo, Liang Chen, and Weiqiang Wang. UI-Venus Technical Report: Building High-performance UI Agents with RFT, 2025. 2
- [9] Yuan Guo, Tingjia Miao, Zheng Wu, Pengzhou Cheng, Ming Zhou, and Zhuosheng Zhang. Atomic-to-Compositional Generalization for Mobile Agents with A New Benchmark and Scheduling System, 2025. 3
- [10] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. CogAgent: A Visual Language Model for GUI Agents, 2024. 2
- [11] Yi Kong, Dianxi Shi, Guoli Yang, Zhang ke-di, Chenlin Huang, Xiaopeng Li, and Songchang Jin. MapAgent: Trajectory-Constructed Memory-Augmented Planning for Mobile Task Automation, 2025. 3
- [12] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. ComputerRL: Scaling End-to-End Online Reinforcement Learning for Computer Use Agents, 2025. 1
- [13] Jiahao Li and Kaer Huang. A Survey on GUI Agents with Foundation Models Enhanced by Reinforcement Learning, 2025. 2
- [14] Ning Li, Qiqiang Lin, Zheng Wu, Xiaoyun Mo, Weiming Zhang, Yin Zhao, Xiangmou Qu, Jiamu Zhou, Jun Wang, Congmin Zheng, Yuanyi Song, Hongjiang Chen, Heyuan Huang, Jihong Wang, Jiaxin Yin, Jingwei Yu, Junwei Liao,

- Qiuying Peng, Xingyu Lou, Jun Wang, Weiwen Liu, Zhuosheng Zhang, and Weinan Zhang. ColorAgent: Building A Robust, Personalized, and Interactive OS Agent, 2025. 1, 2
- [15] Ning Li, Xiangmou Qu, Jiamu Zhou, Jun Wang, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, Jun Wang, and Weinan Zhang. MobileUse: A GUI Agent with Hierarchical Reflection for Autonomous Mobile Operation, 2025. 3
- [16] Weizhen Li, Jianbo Lin, Zhuosong Jiang, Jingyi Cao, Xinpeng Liu, Jiayu Zhang, Zhenqiang Huang, Qianben Chen, Weichen Sun, Qiexiang Wang, Hongxuan Lu, Tianrui Qin, Chenghao Zhu, Yi Yao, Shuying Fan, Xiaowan Li, Tiannan Wang, Pai Liu, King Zhu, He Zhu, Dingfeng Shi, PiaoHong Wang, Yeyi Guan, Xiangru Tang, Minghao Liu, Yuchen Eleanor Jiang, Jian Yang, Jiaheng Liu, Ge Zhang, and Wangchunshu Zhou. Chain-of-Agents: End-to-End Agent Foundation Models via Multi-Agent Distillation and Agentic RL, 2025. 1
- [17] Yanda Li, Chi Zhang, Wenjia Jiang, Wanqi Yang, Bin Fu, Pei Cheng, Xin Chen, Ling Chen, and Yunchao Wei. AppAgent v2: Advanced Agent for Flexible Mobile Interactions, 2025. 3
- [18] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. InfGUI-R1: Advancing Multimodal GUI Agents from Reactive Actors to Deliberative Reasoners, 2025. 2
- [19] Quanfeng Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices, 2024. 7
- [20] Quanfeng Lu, Zhantao Ma, Shuai Zhong, Jin Wang, Dahai Yu, Michael K. Ng, and Ping Luo. SWIRL: A Staged Workflow for Interleaved Reinforcement Learning in Mobile GUI Control, 2025. 3, 7
- [21] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning, 2025. 1, 2
- [22] Zhengxi Lu, Jiabo Ye, Fei Tang, Yongliang Shen, Haiyang Xu, Ziwei Zheng, Weiming Lu, Ming Yan, Fei Huang, Jun Xiao, et al. Ui-s1: Advancing gui automation via semi-online reinforcement learning. *arXiv preprint arXiv:2509.11543*, 2025.
- [23] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. GUI-R1: A Generalist R1-Style Vision-Language Action Model For GUI Agents, 2025. 1, 2, 3, 7
- [24] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering Automated GUI Interaction with Native Agents, 2025. 1
- [25] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillcrap, and Oriana Riva. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents, 2025. 1, 2
- [26] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024. 2, 5
- [27] Yucheng Shi, Wenhao Yu, Zaitang Li, Yonglin Wang, Hongming Zhang, Ninghao Liu, Haitao Mi, and Dong Yu. MobileGUI-RL: Advancing Mobile GUI Agent through Reinforcement Learning in Online Environment, 2025. 2, 3
- [28] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5555–5579, 2025. 1, 2
- [29] Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. SEAgent: Self-Evolving Computer Use Agent with Autonomous Learning from Experience, 2025. 2, 3
- [30] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, et al. Gui-g²: Gaussian reward modeling for gui grounding. *arXiv preprint arXiv:2507.15846*, 2025. 3
- [31] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, Kaitao Song, Jian Shao, Weiming Lu, Jun Xiao, and Yueting Zhuang. A survey on (m)llm-based gui agents, 2025. 1
- [32] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, Wanjun Zhong, Yining Ye, Yujia Qin, Yuwen Xiong, Yuxin Song, Zhiyong Wu, Aoyan Li, Bo Li, Chen Dun, Chong Liu, Daoguang Zan, Fuxing Leng, Hanbin Wang, Hao Yu, Haobin Chen, Hongyi Guo, Jing Su, Jingjia Huang, Kai Shen, Kaiyu Shi, Lin Yan, Peiyao Zhao, Pengfei Liu, Qinghao Ye, Renjie Zheng, Shulin Xin, Wayne Xin Zhao, Wen Heng, Wenhao Huang, Wenqian Wang, Xiaobo Qin, Yi Lin, Youbin Wu, Zehui Chen, Zihao Wang, Baoquan Zhong, Xinchun Zhang, Xujing Li, Yuanfan Li, Zhongkai Zhao, Chengquan Jiang, Faming Wu, Haotian Zhou, Jinlin Pang, Li Han, Qi Liu, Qianli Ma, Siyao Liu, Songhua Cai, Wenqi Fu, Xin Liu, Yaohui Wang, Zhi Zhang, Bo Zhou, Guoliang Li, Jiajun Shi, Jiale Yang, Jie Tang, Li Li, Qihua Han, Taoran Lu, Woyu Lin, Xiaokang Tong, Xinyao Li, Yichi Zhang, Yu Miao, Zhengxuan Jiang, Zili Li, Ziyuan Zhao, Chenxin Li, Dehua Ma, Feng Lin, Ge Zhang, Haihua Yang, Hangyu Guo, Hongda Zhu, Jiaheng Liu, Junda Du, Kai Cai, Kuanye Li, Lichen Yuan, Meilan Han, Minchao Wang, Shuyue Guo, Tianhao Cheng, Xiaobo Ma, Xiaojun Xiao, Xiaolong Huang, Xinjie Chen, Yidi Du, Yilin Chen,

- Yiwen Wang, Zhaojian Li, Zhenzhu Yang, Zhiyuan Zeng, Chaolin Jin, Chen Li, Hao Chen, Haoli Chen, Jian Chen, Qinghao Zhao, and Guang Shi. UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning, 2025. [2](#)
- [33] Zhepei Wei, Wenlin Yao, Yao Liu, Weizhi Zhang, Qin Lu, Liang Qiu, Changlong Yu, Puyang Xu, Chao Zhang, Bing Yin, Hyokun Yun, and Lihong Li. WebAgent-R1: Training Web Agents via End-to-End Multi-Turn Reinforcement Learning, 2025. [1](#), [2](#)
- [34] Qinzhuo Wu, Pengzhi Gao, Wei Liu, and Jian Luan. Back-trackAgent: Enhancing GUI Agent with Error Detection and Backtracking Mechanism, 2025. [1](#)
- [35] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents, 2024. [1](#), [2](#), [7](#)
- [36] Zheng Wu, Pengzhou Cheng, Zongru Wu, Lingzhong Dong, and Zhuosheng Zhang. Gem: Gaussian embedding modeling for out-of-distribution detection in gui agents. *arXiv preprint arXiv:2505.12842*, 2025. [3](#)
- [37] Zheng Wu, Heyuan Huang, Xingyu Lou, Xiangmou Qu, Pengzhou Cheng, Zongru Wu, Weiwen Liu, Weinan Zhang, Jun Wang, Zhaoxiang Wang, and Zhuosheng Zhang. VeriOS: Query-Driven Proactive Human-Agent-GUI Interaction for Trustworthy OS Agents, 2025. [1](#)
- [38] Zhe Wu, Hongjin Lu, Junliang Xing, Changhao Zhang, Yin Zhu, Yuhao Yang, Yuheng Jing, Kai Li, Kun Shao, Jianye Hao, Jun Wang, and Yuanchun Shi. Hi-Agent: Hierarchical Vision-Language Agents for Mobile Device Control, 2025. [3](#)
- [39] Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. MobileRL: Online Agentic Reinforcement Learning for Mobile GUI Agents, 2025. [2](#)
- [40] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguviz: Unified Pure Vision Agents for Autonomous GUI Interaction, 2025. [1](#), [2](#)
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, 2025. [7](#)
- [42] Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, Wenhai Wang, Yu Qiao, Xizhou Zhu, and Jifeng Dai. ZeroGUI: Automating Online GUI Learning at Zero Human Cost, 2025. [2](#)
- [43] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025. [2](#), [3](#)
- [44] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279*, 2024. [2](#)
- [45] Cheng Zhang, Erhu Feng, Xi Zhao, Yisheng Zhao, Wangbo Gong, Jiahui Sun, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. MobiAgent: A Systematic Framework for Customizable Mobile Agents, 2025. [3](#)
- [46] Jiwen Zhang, Jihao Wu, Yihua Teng, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the Zoo: Chain-of-Action-Thought for GUI Agents, 2024. [7](#)
- [47] Zijing Zhang, Ziyang Chen, Mingxiao Li, Zhaopeng Tu, and Xiaolong Li. RLVMR: Reinforcement Learning with Verifiable Meta-Reasoning Rewards for Robust Long-Horizon Agents, 2025. [1](#), [2](#)
- [48] Zhong Zhang, Yaxi Lu, Yikun Fu, Yupeng Huo, Shenzhi Yang, Yesai Wu, Han Si, Xin Cong, Haotian Chen, Yankai Lin, Jie Xie, Wei Zhou, Wang Xu, Yuanheng Zhang, Zhou Su, Zhongwu Zhai, Xiaoming Liu, Yudong Mei, Jianming Xu, Hongyan Tian, Chongyi Wang, Chi Chen, Yuan Yao, Zhiyuan Liu, and Maosong Sun. AgentCPM-GUI: Building Mobile-Use Agents with Reinforcement Fine-Tuning, 2025. [1](#)